

Guia turístico simples - Flutter

Guia Turístico de Wiesbaden



 Ver Pontos de Interesse

 Meus Favoritos

Autores

David Martins Correia – 2023140410

Tiago Gouveia Filipe – 2019112767

Coimbra, 24/12/2025



**Instituto Superior
de Engenharia**

Politécnico de Coimbra

ÍNDICE:

Índices de figuras	1
Introdução	2
Arquitetura do Sistema	3
Diagrama de Entidades (Lógica de Dados)	4
Fluxo de Dados da Meteorologia	5
Justificação Técnica da Persistência (Shared Preferences)	6
Conclusão.....	7
Referências.....	8

ÍNDICES DE FIGURAS

Figura 1 - Lista de Interesses.....	4
Figura 2 - Apresentação de Ponto Turístico	5
Figura 3 - Adicionar aos Favoritos.....	6
Figura 4 - Lista Favoritos.....	6

INTRODUÇÃO

O presente relatório descreve o desenvolvimento de uma aplicação móvel turística dedicada à cidade de Wiesbaden, na Alemanha, realizada no âmbito da unidade curricular de Arquiteturas de Móveis. O objetivo principal do projeto consiste na criação de um guia digital interativo que permita aos utilizadores explorar os principais Pontos de Interesse (POIs) da cidade, organizados por categorias temáticas.

A aplicação foi desenvolvida utilizando a framework Flutter e a linguagem de programação Dart, tirando partido da sua arquitetura reativa e da capacidade de gerar interfaces nativas de elevada performance. Para além da apresentação de informações estáticas, o projeto integra funcionalidades como:

- Consumo de Dados Locais: Gestão de informação estruturada através de ficheiros JSON.
- Integração de APIs Externas: Obtenção de dados meteorológicos em tempo real via Open-Meteo API.
- Persistência de Dados: Implementação de um sistema de "Favoritos" que mantém as preferências do utilizador gravadas no dispositivo através de Shared Preferences.
- Interfaces Responsivas: Adaptação do layout para diferentes orientações de ecrã (Portrait e Landscape), garantindo uma experiência de utilizador (UX) consistente.

Ao longo deste relatório, serão detalhadas as decisões arquiteturais, as tecnologias implementadas e as soluções encontradas para os desafios técnicos propostos, demonstrando a aplicação prática dos conceitos de computação móvel explorados durante as aulas.

ARQUITETURA DO SISTEMA

A aplicação foi desenvolvida seguindo os princípios de modularidade e separação de conceitos (Separation of Concerns), garantindo a escalabilidade e facilidade de manutenção do código.

4.1. Modelo de Dados (Models) Foi adotada uma estrutura de dados hierárquica baseada no ficheiro JSON de exemplo. A classe de topo, `CityData`, encapsula a lógica de parsing para as classes `Category` e `Poi` (Point of Interest). Esta abordagem permite que a aplicação suporte múltiplos pontos de interesse organizados por categorias (Monumentos, Cultura, Natureza, etc.), facilitando a navegação do utilizador.

4.2. Camada de Serviços (Services) A lógica de negócio foi isolada em serviços estáticos:

DataService: Responsável pela leitura assíncrona do ficheiro JSON local, transformando dados em objetos Dart.

WeatherService: Gere a comunicação externa com a API REST **Open-Meteo**

PrefsService: Implementa a persistência local utilizando o plugin `shared_preferences`. Esta camada permite que as preferências do utilizador (Favoritos) sejam mantidas entre sessões, guardando apenas os identificadores únicos (IDs) dos monumentos.

4.3. Interface e Navegação (UI) A interface foi construída utilizando uma combinação de:

StatelessWidgets: Para ecrãs puramente informativos e de listagem.

StatefulWidgets: Para o ecrã de detalhes, permitindo a atualização dinâmica da interface (ex: ícone de favorito e carregamento de meteorologia via `FutureBuilder`).

Navigator (Stack Navigation): Utilizado para gerir o fluxo entre o ecrã principal, a lista de categorias e os detalhes, garantindo uma experiência de utilizador fluida e intuitiva.

4.4. Robustez e Tratamento de Erros A aplicação foi desenhada para ser resiliente a falhas:

Uso de `errorBuilder` em imagens para prevenir quebras visuais caso os assets falhem.

Tratamento de exceções em chamadas HTTP para evitar o fecho inesperado da app em caso de falta de conectividade.

Verificação do estado mounted antes de atualizar o estado em operações assíncronas.

DIAGRAMA DE ENTIDADES (LÓGICA DE DADOS)

Para o desenvolvimento do projeto seguindo o exemplo do json partilhado com o enunciado foi desenvolvido o diagrama de entidades.

- **CityData** (Wiesbaden)
 - **Category** (Monumentos, Natureza, etc.)
 - **POI** (Nome, Coordenadas, Preço, etc.)

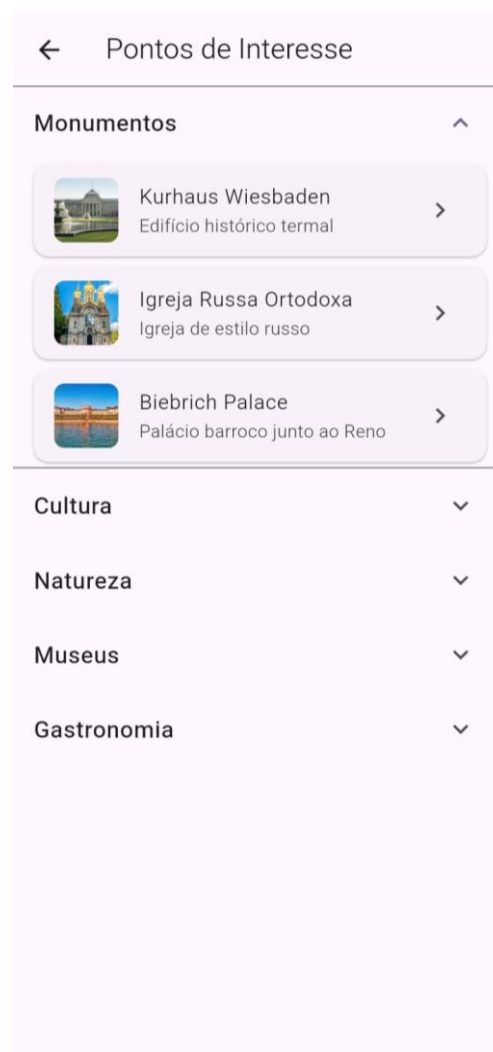


Figura 1 - Lista de Interesses

FLUXO DE DADOS DA METEOROLOGIA

O fluxo de obtenção da meteorologia segue um padrão assíncrono não-bloqueante: ao iniciar o ecrã, o identificador do Poi fornece as coordenadas (latitude, longitude) ao WeatherService, que interroga a API Open-Meteo. Durante este processo, a UI permanece responsiva, apresentando um indicador de progresso (CircularProgressIndicator) até que o Future seja resolvido.



Figura 2 - Apresentação de Ponto Turístico

JUSTIFICAÇÃO TÉCNICA DA PERSISTÊNCIA (SHARED PREFERENCES)

O uso de shared_preferences em detrimento de uma base de dados relacional (SQLite) devido à natureza leve dos dados a persistir (apenas IDs de favoritos). Esta escolha otimiza o consumo de recursos e simplifica a lógica de I/O, mantendo a performance ideal para dispositivos de gama média/baixa.



Figura 3 - Adicionar aos Favoritos



Figura 4 - Lista Favoritos

CONCLUSÃO

O desenvolvimento desta aplicação permitiu a consolidação de conhecimentos fundamentais no domínio das arquiteturas móveis. A utilização da framework **Flutter** revelou-se uma escolha eficaz, possibilitando a criação de uma interface rica e fluida, enquanto a linguagem **Dart** facilitou a gestão de processos assíncronos, essenciais no consumo de recursos externos.

Ao longo do projeto, foram superados desafios técnicos importantes, nomeadamente:

- A gestão eficiente do ciclo de vida dos widgets, otimizando o carregamento de dados da API para evitar chamadas redundantes à rede.
- A garantia de persistência de dados local, oferecendo uma funcionalidade de "Favoritos".

Em suma, a aplicação cumpre todos os requisitos propostos, apresentando-se como uma ferramenta funcional, robusta e responsiva. Este projeto serviu não só para aplicar os conceitos lecionados, mas também para explorar boas práticas de desenvolvimento, como o tratamento de erros e a otimização da experiência do utilizador (UX).

REFERÊNCIAS

Flutter Documentation (Official): *Reference Documentation*. Disponível em: <https://docs.flutter.dev> (Acedido em Dezembro de 2025).



**Instituto Superior
de Engenharia**

Politécnico de Coimbra